BloodConnect

Technical Guide, Code Reference and Deployment Instructions

Prepared from the application source in this workspace. This guide documents the implementation as it exists: React frontend, Spring Boot API, PostgreSQL schema, JWT security and Docker Compose startup.

Scope note: This is an engineering guide, not a medical protocol. Blood compatibility and operational decisions should be reviewed by qualified clinical and regulatory teams.

1. System overview

BloodConnect is an emergency blood donation coordination platform. It lets users register, create or track blood requests, create donor profiles, receive donor requests and notifications, and supports administrative donor review. The frontend runs as a Vite/React single-page application. The backend is a Spring Boot REST API with PostgreSQL persistence.

Client	React 18, Vite, React Router, Zustand, Axios	Authentication state, route protection, donor and request screens
API	Java 17, Spring Boot 3.1.5	REST controllers, validation, business rules, security
Data	PostgreSQL 15, JPA, Flyway	Users, profiles, requests, notifications and audit records
Infrastructure	Docker Compose, Nginx frontend container	Local multi-container startup and service health checks

Request flow: a requester submits a blood request. The API stores it, asks the matching service for compatible verified and available donors within the configured radius, creates donor-request records, and makes notifications available to the matched donors.

2. Repository map

The codebase separates the browser application from the server application. This is useful because frontend concerns such as routes and forms remain independent from persistence and authorization code.

Frontend entry points

```
frontend/src/App.jsx                Route tree and auth-aware navigation
frontend/src/pages/                 Screen components
frontend/src/services/api.js         Axios API client
frontend/src/stores/authStore.js     Zustand authentication state
frontend/src/App.css, index.css      Shared styling
```

Backend entry points

```
backend/src/main/java/com/bloodconnect/api/controller/  HTTP endpoints
backend/src/main/java/com/bloodconnect/service/         Business logic
backend/src/main/java/com/bloodconnect/domain/entity/   JPA entities
backend/src/main/java/com/bloodconnect/repository/      Data access
backend/src/main/resources/db/migration/                Flyway SQL migrations
backend/src/main/resources/application.yml              Runtime configuration
```

3. Frontend routing and state

App.jsx hydrates the persisted authentication store when the application starts. Routes for login and registration are shown when no user exists; authenticated users receive the home, request, donor and admin routes. This route split prevents the browser from rendering private pages before a user session is available.

Auth-aware route structure - frontend/src/App.jsx

```
const { user, hydrate } = useAuthStore();

useEffect(() => {
  hydrate();
}, [hydrate]);

{!user ? (
  <Route path="/login" element={<Login />} />
) : (
  <Route path="/create-request" element={<CreateBloodRequest />} />
)}
```

Implementation instruction: keep API calls in services/api.js rather than duplicating Axios configuration in page components. Keep session-specific values in authStore.js and clear them on logout. When adding a new

authenticated feature, add both a page component and a protected route in App.jsx.

4. Backend API and authentication

Controllers use Spring MVC annotations to expose endpoints. AuthController accepts registration, login and token-refresh requests. BloodRequestController reads the authenticated user ID placed on the request by the JWT filter, then delegates all domain work to BloodRequestService.

Login endpoint - backend/.../controller/AuthController.java

```
@PostMapping("/login")
public ResponseEntity<AuthResponse> login(
    @Valid @RequestBody AuthRequest request) {
  AuthResponse response = authService.login(request);
  return ResponseEntity.ok(response);
}
```

Create request endpoint - backend/.../controller/BloodRequestController.java

```
@PostMapping
public ResponseEntity<BloodRequestResponse> createBloodRequest(
    @Valid @RequestBody CreateBloodRequestDTO request,
    HttpServletRequest httpRequest) {
  Long userId = (Long) httpRequest.getAttribute("userId");
  BloodRequestResponse response =
      bloodRequestService.createBloodRequest(request, userId);
  return ResponseEntity.status(HttpStatus.CREATED).body(response);
}
```

Security instruction: set a unique JWT_SECRET outside source control for every environment. The checked-in defaults support only local development. Use HTTPS in production, restrict allowed CORS origins to deployed client domains, and consider rotating refresh tokens and adding rate limiting to login endpoints.

5. Donor matching logic

MatchingService stores a compatibility map keyed by requested blood group. It selects donors whose group appears in the allowed set, whose profiles are VERIFIED and AVAILABLE, and whose coordinates fall inside the request radius. It then sorts candidates by distance.

Distance filter - backend/.../service/MatchingService.java

```
return donors.stream()
  .filter(donor -> donor.getLatitude() != null
               && donor.getLongitude() != null)
  .filter(donor -> calculateDistance(latitude, longitude,
       donor.getLatitude(), donor.getLongitude()) <= radiusKm)
  .sorted((d1, d2) -> Double.compare(
       calculateDistance(latitude, longitude, d1.getLatitude(), d1.getLongitude()),
       calculateDistance(latitude, longitude, d2.getLatitude(), d2.getLongitude())))
  .collect(Collectors.toList());
```

The distance calculation uses the Haversine formula with an Earth radius of 6,371 km. For larger donor volumes, move the radius search into the database using a geospatial index or PostGIS; the current service fetches qualifying donor groups first and filters coordinates in application memory.

6. Database design

Flyway migration V1 defines the core relational model. users is the identity table. donor_profiles and hospitals extend a user account. blood_requests belongs to a requester and optionally a hospital. donor_requests provides the many-to-many record that captures a donor notification and response. notifications and audit_logs support user communication and traceability.

Table	Purpose	Important relationships
users	Account identity and role	Parent of donor_profiles, hospitals, requests, notifications

donor_profiles	Donor eligibility, blood group, availability and location	One profile per user
blood_requests	Demand for blood units	Requester user; optional hospital
donor_requests	Match and response record	Unique request + donor pair
notifications	User-visible event messages	Related entity fields link to request or match
audit_logs	Operational activity history	Optional user reference

Schema instruction: create later schema changes as new ordered Flyway migrations. Do not edit a migration that has already been applied in a shared environment.

7. Local setup and launch

Docker Compose is the quickest way to start the three application services. It provisions PostgreSQL, waits for its health check, builds the Spring API, then serves the frontend. The browser-facing frontend port is 3000; API requests use the /api context path and the backend health endpoint is /api/auth/health.

Run locally

```
# From the repository root
docker compose up --build

# Open the frontend
http://localhost:3000

# Check the API health response
http://localhost:8080/api/auth/health
```

For frontend-only work, run npm install and npm run dev inside frontend. For backend-only work, configure a PostgreSQL database through DB_URL, DB_USERNAME, DB_PASSWORD and JWT_SECRET, then run Maven from backend. The default application.yml listens on port 8080 with a /api context path.

8. Verification and next steps

Before publishing, validate API behavior with unit and integration tests, test role restrictions on every protected endpoint, and test request matching using known locations and blood groups. Confirm errors use consistent response shapes. Review production secrets, database backups, audit-log retention, monitoring and alerting before handling real donor data.

Recommended engineering improvements

1. Add integration tests for registration, login, JWT rejection and blood-request creation.
2. Add database-level or geospatial support for scalable nearby-donor searches.
3. Add API documentation with OpenAPI and examples for every controller.
4. Add an environment-specific CORS allowlist and secrets manager integration.
5. Have clinical and legal owners validate the compatibility and donor eligibility rules before production use.